Concurrent Dispatch for Home Energy Systems

Daniel Murong — Intro to Distributed Systems, Spring 2026

1. Introduction

A home with rooftop solar panels, a home battery, and a connection to the electric grid must have a process of routing energy among them and the household load. Spontaneous load spikes can occur, and energy prices from the grid vary throughout the day. This project implements a Go simulator of such a home, where each subsystem is a **goroutine** communicating over **channels**, and uses it to compare three battery-management policies under Duke Energy's NC Residential Solar TOU rates.

Thesis: Greedy single-objective policies are unreliable across seasons. Each naive baseline beats the other in one season and loses in the other. A two-parameter policy beats both. These results provide an analogy to how production auto-scalers like the Kubernetes Horizontal Pod Autoscaler and AWS Auto Scaling Groups are tuned.

2. System Architecture

In this simulation, each subsystem has its own goroutine and only communicates with the others through typed messages over buffered channels. The topology is the following: `Clock --Tick--> {Solar, Load} --> Coordinator --Decision--> Battery --> Logger`. A **Clock** goroutine fires a `Tick` every simulated 5 minutes (288 a day), carrying the logical timestamp and details about the grid. **Solar** and **Load** goroutines each read a copy of every tick from channels fanned out from the `clockCh` channel and produce independent kW values. The **coordinator** represents the loop that applies the active **policy** to the received outcomes from **Solar** and **Load**. A `Decision` is made describing how to route energy across six possible source-sink pairs. That decision goes to updating the battery state, and the full tick record is written to CSV using the **logger**. This mirrors a real distributed control plane: sensors, actuators, and a controller as independent processes over a message bus.

Inputs are calibrated against the Duke Energy Schedule RSTC, NREL TMY3, the Tesla Powerwall 2 datasheet, and EIA RECS + NREL ResStock.

3. Policies Under Test

Policy	Rule
Greedy Cheap-Fill	Grid-charge to 100% during off-peak hours.
Greedy Drain	Drain to 0% during on-peak hours.
Two-Threshold	Maintain reserve floor (20%) and charge ceiling (80%).

4. Results

Each policy ran for one simulated day under summer and non-summer tariffs, with identical solar production and load profiles across all three by using the same seed. Duke's on-peak window differs by season: **summer is 3–6 PM**, which overlaps with the declining tail of solar production, and **non-summer is 6–9 AM**, which happens entirely before the sun comes up. That one difference is what changes the results across seasons.

Season	Policy	Net Cost	Solar Util.	Min SOC	Spilled
Summer	**Two-Threshold**	**$1.62**	63.8%	53.1%	15.9 kWh
Summer	Greedy Cheap-Fill	$2.26	57.6%	53.1%	18.6 kWh
Summer	Greedy Drain	$2.63	36.6%	16.6%	27.9 kWh
Non-summer	**Two-Threshold**	**$2.03**	50.0%	30.6%	15.2 kWh
Non-summer	Greedy Drain	$2.30	37.0%	33.7%	19.1 kWh
Non-summer	Greedy Cheap-Fill	$3.17	35.3%	53.1%	19.7 kWh

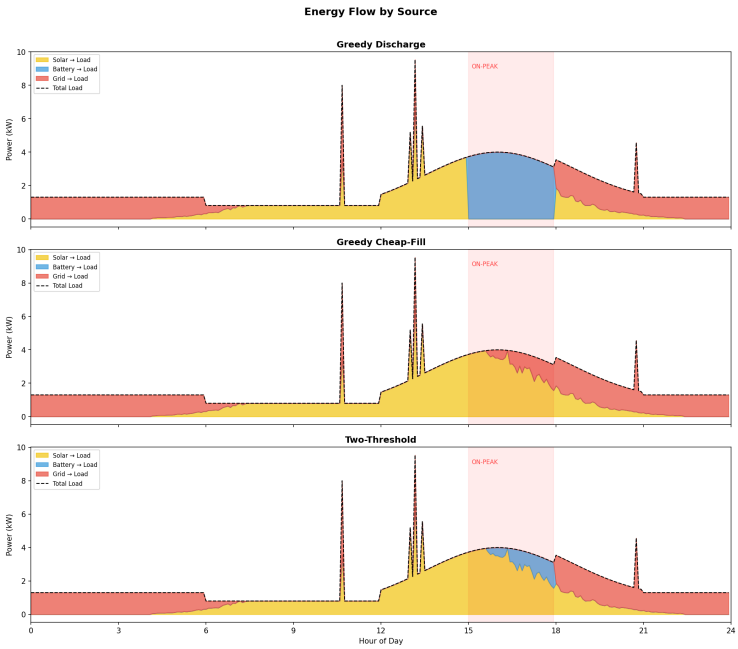

Figure 1: Summer energy flows by policy. Yellow = solar→load, blue = battery→load, red = grid→load; shaded band = 3–6 PM on-peak window.

In summer, cheap-fill beats drain by $0.37. In non-summer, drain beats cheap-fill by $0.87. Two-threshold wins both, by 28% in summer and 12% in non-summer over the next-best greedy of that season.

Why the flip: In summer (Fig. 1), cheap-fill has filled the battery overnight and hits 100% by early morning. When midday solar arrives, there's nowhere to put it, so it spills to the grid at $0.03/kWh instead of displacing $0.098–$0.224 in grid purchases. Drain does the opposite: it empties the battery aggressively during on-peak but also routes solar to the grid at $0.03 while doing it since the policy prioritizes battery discharge over solar self-consumption. It ends up at 17% SOC mid-peak all the while throwing away free solar the whole time.

Non-summer reverses the outcome. The 6–9 AM peak arrives before any solar energy is captured, so draining the battery then is rational since there's no storage opportunity cost. Cheap-fill, on the other hand, still never drains, so the entire morning peak is served from the grid at $0.224/kWh with a full battery sitting idle. Two-threshold is the only policy that both covers the peak from the battery *and* leaves overnight headroom for midday solar.

5. Discussion: Structural Slack as a Distributed-Systems Primitive

Two-threshold uses two parameters, each preventing a failure point: a **reserve floor (20%)** that guarantees backup capacity, and a **charge ceiling (80%)** that preserves headroom for incoming solar. What is notable is that **neither threshold does any predicting**. The policy has no idea what season it is, when the sun will rise, or how large the afternoon load will be. It just refuses to commit capacity at either extreme, and that structural constraint alone is enough to beat both greedy baselines in both seasons.

The same principle shows up in production auto-scalers. *Greedy cheap-fill* maps onto **over-provisioning**: pre-scaling ahead of anticipated demand causes the capacity to sit idle, and any spike with work that exceeds the pre-scaled amount still requires on-demand premium rates to fill the gap. *Greedy drain* maps onto **aggressive scale-down**: min-replicas set too low can cause cold starts, pool exhaustion, and cascading timeouts on the next spike. *Two-threshold* is comparable to the **K8s HPA utilization band** and **AWS ASG `MinSize`/`MaxSize`**: the lower bound offers insurance for sudden spikes in work while the upper bound limits blast radius and wasteful spending on excess capacity, and neither require forecasting.

6. Limitations & Reproduction

This is a single-day, single-seed simulation with a parametric solar curve. Round-trip battery efficiency, the $14/mo customer charge on Schedule RSTC, and weekend tariffs are not modeled. AI assistance (Anthropic Claude) was used for code scaffolding and editorial revision; the modeling and analysis are my own.